

documentación

Actores

- Cliente: Puede registrarse, iniciar sesión, navegar el catálogo, filtrar por categoría, gestionar su wishlist, realizar pedidos y dejar reseñas en productos comprados.
- Administrador: Puede iniciar sesión, navegar el catálogo, gestionar (CRUD) productos y categorías, y actualizar el estado de los pedidos.

Requisitos Funcionales

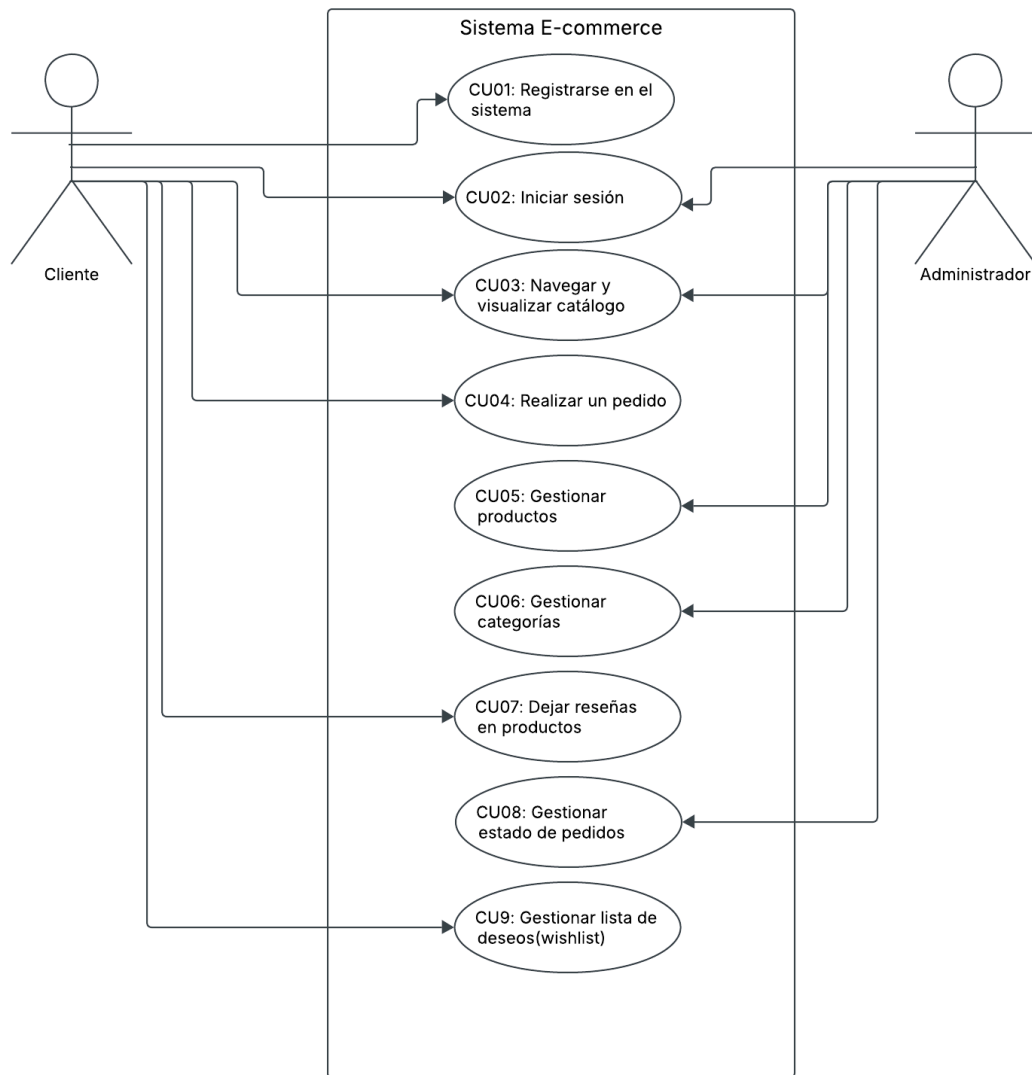
- RF01: El sistema debe permitir el registro de usuarios.
- RF02: El sistema debe permitir a los usuarios iniciar sesión.
- RF03: El sistema debe mostrar el catálogo de productos sin TACC.
- RF04: El sistema debe permitir filtrar productos por categoría.
- RF05: El sistema debe permitir agregar y eliminar productos de la wishlist.
- RF06: El sistema debe permitir realizar pedidos.
- RF07: El sistema debe permitir visualizar pedidos realizados por el usuario.
- RF08: El sistema debe permitir dejar reseñas en productos comprados.
- RF09: El administrador debe poder crear, editar y eliminar productos.
- RF10: El administrador debe poder gestionar el estado de los pedidos.
- RF11: El sistema debe calcular automáticamente el total del pedido.
- RF12: El administrador debe poder crear, editar y eliminar categorías.

Requisitos No Funcionales

- RNF01: La aplicación debe ser responsive.
- RNF02: El sistema debe responder en menos de 3 segundos en operaciones comunes.
- RNF03: Los errores deben mostrarse de forma clara al usuario.
- RNF04: El código debe estar documentado y organizado siguiendo el patrón MVC.
- RNF05: El sistema debe ser usable en dispositivos móviles.

Casos de Uso

- CU01: Registrarse en el sistema
- CU02: Iniciar sesión
- CU03: Navegar y visualizar catálogo de productos
- CU04: Realizar un pedido
- CU05: Gestionar productos (Administrador)
- CU06: Gestionar categorías (Administrador)
- CU07: Dejar reseñas en productos
- CU08: Gestionar estado de pedidos (Administrador)
- CU09: Gestionar lista de deseos (Wishlist)



Alcance y Supuestos

- No se implementará pasarela de pago real.
- No se gestionarán envíos reales.
- El sistema funcionará como simulación de ecommerce.
- Se asumirá que todos los productos son aptos sin TACC.

Modelo de datos y estructura de la base de datos

La aplicación utiliza Laravel Eloquent ORM para la interacción con la base de datos. La estructura del sistema fue definida mediante migraciones de Laravel, utilizando claves foráneas, restricciones de integridad referencial y relaciones entre modelos.

Las entidades principales del sistema son:

- Usuarios
- Categorías

- Productos
- Pedidos
- Detalle de pedidos
- Reseñas
- Lista de deseos
- Direcciones

Migraciones

Las migraciones permiten definir y versionar la estructura de la base de datos. Cada tabla fue creada utilizando las herramientas proporcionadas por Laravel, evitando consultas SQL directas.

Se utilizaron:

- Claves primarias mediante ``id()``.
- Claves foráneas mediante ``foreignId()->constrained()``.
- Eliminación en cascada mediante ``onDelete('cascade')``.
- Tipos de datos adecuados según la información almacenada.
- Restricciones para garantizar la integridad de los datos.

Tabla usuarios

La tabla ``usuarios`` representa a los usuarios registrados dentro del sistema.

Esta entidad contempla tanto clientes como administradores, diferenciados mediante el atributo ``es_admin``.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador único del usuario
nombre	string	Nombre del usuario
email	string	Correo electrónico único
password	string	Contraseña almacenada de forma segura
es_admin	boolean	Define si el usuario posee permisos administrativos
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de actualización

Roles

El sistema maneja dos tipos de usuarios:

- Cliente: `es\_admin = false`
- Administrador: `es\_admin = true`

El modelo `Usuario` extiende de `Authenticatable`, permitiendo utilizar el sistema de autenticación proporcionado por Laravel.

Tabla categories

La tabla `categories` almacena las categorías disponibles para clasificar los productos.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador de categoría
name	string	Nombre de la categoría

Además, la tabla utiliza eliminación lógica mediante `SoftDeletes`, permitiendo ocultar categorías sin eliminarlas físicamente.

Tabla products

La tabla `products` representa el catálogo de productos disponibles en la tienda.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador del producto
name	string	Nombre del producto
description	text	Descripción del producto
price	decimal	Precio del producto
stock	integer	Cantidad disponible
image	string	Ruta de la imagen
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de actualización

El precio se almacena utilizando tipo decimal para mantener precisión en valores monetarios.

Tabla category_product

Esta tabla funciona como tabla intermedia entre categorías y productos.

Se utiliza debido a que existe una relación muchos a muchos:

Categoría N ----- M Producto

Un producto puede pertenecer a varias categorías y una categoría puede contener múltiples productos.

Campos

Campo	Tipo
product_id	foreign key
category_id	foreign key

Las claves foráneas utilizan eliminación en cascada:

```
`onDelete('cascade')`
```

Esto permite eliminar automáticamente las relaciones asociadas cuando se elimina una categoría o producto.

Tabla orders

La tabla **orders** almacena los pedidos realizados por los usuarios.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador del pedido
usuario_id	foreign key	Usuario que realizó el pedido
total	decimal	Importe total del pedido
status	enum	Estado actual del pedido
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de actualización

El campo **status** utiliza valores controlados:

- pendiente
- pagado
- enviado
- entregado
- cancelado

Esto permite validar los estados posibles del pedido y evitar valores incorrectos.

Tabla order_items

La tabla `order_items` representa el detalle de cada pedido.

Se utiliza como entidad intermedia porque la relación entre pedidos y productos necesita almacenar información adicional.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador
order_id	foreign key	Pedido asociado
product_id	foreign key	Producto asociado
quantity	integer	Cantidad comprada
unit_price	decimal	Precio del producto al momento de compra

Relación:
Pedido N ----- M Producto

La tabla permite guardar el precio histórico del producto, incluso si posteriormente cambia su precio en el catálogo.

Tabla reviews

La tabla `reviews` almacena las reseñas realizadas por los usuarios sobre productos.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador
usuario_id	foreign key	Usuario que realizó la reseña
product_id	foreign key	Producto reseñado
rating	integer	Calificación otorgada
comment	text	Comentario del usuario

Relaciones:

Usuario 1 ----- N Reseñas

Producto 1 ----- N Reseñas

Tabla wishlists

La tabla **wishlists** representa la lista de deseos de cada usuario.

Permite almacenar productos favoritos.

Campos

Campo	Tipo
usuario_id	foreign key
product_id	foreign key

La tabla posee una clave primaria compuesta:

```
primary(['usuario_id', 'product_id'])
```

Esto evita que un usuario pueda agregar repetidamente el mismo producto a favoritos.

Relación:

Usuario N ----- M Producto

Tabla addresses

La tabla `addresses` almacena las direcciones de envío asociadas a los usuarios.

Campos

Campo	Tipo	Descripción
id	bigint	Identificador
usuario_id	foreign key	Usuario propietario
address	string	Dirección
city	string	Ciudad
province	string	Provincia
postal_code	string	Código postal

Relación:

Usuario 1 ----- N Direcciones

Un usuario puede registrar más de una dirección.

Modelos Eloquent

Cada tabla posee un modelo Laravel asociado, encargado de representar la entidad dentro de la aplicación.

Los modelos implementan:

- Relaciones Eloquent.
- Validación de asignación masiva mediante `$fillable`.
- Conversión automática de tipos mediante `casts`.
- Métodos auxiliares para lógica específica.

Relaciones entre modelos

Usuario

Relaciones:

`hasMany(Order::class)`

hasMany(Review::class)
hasMany(Address::class)
belongsToMany(Product::class)

Un usuario puede tener:

- Muchos pedidos.
- Muchas reseñas.
- Muchas direcciones.
- Muchos productos favoritos.

Producto

Relaciones:

belongsToMany(Category::class)
hasMany(OrderItem::class)
hasMany(Review::class)
belongsToMany(Usuario::class)

Un producto puede:

- Pertenecer a varias categorías.
- Aparecer en múltiples pedidos.
- Tener múltiples reseñas.
- Estar agregado a listas de deseos.

Categoría

Relación:

belongsToMany(Product::class)

Una categoría contiene múltiples productos.

Pedido

Relaciones:

belongsTo(Usuario::class)
hasMany(OrderItem::class)

Un pedido pertenece a un usuario y posee varios productos mediante sus detalles.

OrderItem

Relaciones:

```
belongsTo(Order::class)
belongsTo(Product::class)
```

Cada detalle pertenece a un pedido y referencia un producto.

Review

Relaciones:

```
belongsTo(Usuario::class)
belongsTo(Product::class)
```

Cada reseña pertenece a un usuario y a un producto.

Address

Relación:

```
belongsTo(Usuario::class)
```

Cada dirección pertenece a un usuario.

Casts utilizados

Se implementaron conversiones automáticas mediante Eloquent Casts.

Ejemplos:

Producto

```
'price' => 'decimal:2',
'stock' => 'integer'
```

Permite manejar correctamente precios y cantidades.

Pedido

```
'status' => OrderStatus::class,
'total' => 'decimal:2'
```

El estado del pedido se maneja mediante un Enum de Laravel.

Usuario

```
'es_admin' => 'boolean',
'password' => 'hashed'
```

Permite manejar correctamente permisos y almacenamiento seguro de contraseñas.

Buenas prácticas aplicadas

Durante el desarrollo del modelo de datos se aplicaron las siguientes prácticas:

- Uso de migraciones para controlar cambios en la base de datos.
- Uso de Eloquent ORM en lugar de consultas SQL manuales.
- Definición de relaciones mediante métodos Eloquent.
- Uso de claves foráneas con restricciones.
- Uso de eliminación en cascada para mantener integridad.
- Uso de Soft Deletes en categorías.
- Uso de Casts para transformación automática de datos.
- Uso de Enum para controlar estados de pedidos.
- Separación de responsabilidades siguiendo la arquitectura MVC.

Validación de datos (Form Requests)

El sistema utiliza **Form Requests de Laravel** para centralizar la validación de datos y mantener los controladores limpios.

CategoryRequest

Valida la creación y edición de categorías.

- **name**: obligatorio, string, máximo 255 caracteres, único

Evita duplicados en categorías

ProductRequest

Valida productos.

- **name**: obligatorio
- **description**: opcional
- **price**: numérico ≥ 0
- **stock**: entero ≥ 0
- **image**: jpg, png, webp (máx 10MB)
- **categories**: array obligatorio

✓ Controla integridad de productos y categorías

RegisterRequest

Valida registro de usuarios.

- contraseña segura:
 - mínimo 8 caracteres
 - una mayúscula
 - un número
 - un símbolo

También valida dirección del usuario

LoginRequest

Valida inicio de sesión:

- email válido
- password obligatorio

CheckoutProcessRequest

Valida el carrito antes de comprar:

- carrito no vacío
- cantidades válidas

Usa `prepareForValidation()` para traer datos desde sesión

ReviewRequest

Valida reseñas:

- rating: 1 a 5
- comentario obligatorio

UpdateProfileRequest

Permite actualizar:

- nombre
- email (único)
- dirección

✓ Actualiza usuario + dirección en una transacción

UpdatePasswordRequest

Valida cambio de contraseña:

- contraseña actual obligatoria
- nueva contraseña segura

UpdateOrderStatusRequest

Valida estado de pedido:

- **status**: requerido

StoreUsuarioRequest / UpdateUsuarioRequest

Validan creación y edición de usuarios.

CONTROLLERS

Arquitectura

Los controladores siguen el patrón MVC y delegan la lógica a servicios.

AuthController

Gestiona autenticación:

- login
- logout
- registro

✓ Usa **AuthService**

ProductController

CRUD de productos:

- create / store
- edit / update
- delete

✓ Manejo de imágenes

✓ Relación con categorías

CategoryController

CRUD de categorías usando **CategoryService**

CartController

Gestión del carrito:

- agregar productos
- incrementar / decrementar

- eliminar

Usa sesión

CheckoutController

Procesa compra:

- valida carrito
- crea orden

Usa transacciones

OrderController

Pedidos del usuario:

- listar
- ver detalle
- crear (API)

AdminOrderController

Pedidos para admin:

- ver todos
- actualizar estado

Usa **OrderService**

ProfileController

Perfil del usuario:

- editar datos
- cambiar contraseña

Usa transacciones

WishlistController

Favoritos:

- agregar / quitar
- listar

ReviewController

Reseñas:

- crear
- eliminar

Usa Policy

CatalogController / StoreController

Catálogo público:

- filtros por categoría
- filtros por precio

UsuarioController

Gestión de usuarios (admin)

AdminController

Dashboard con estadísticas:

- productos
- categorías
- usuarios

DOCUMENTACIÓN – SERVICES

Capa de servicios

Se implementa una capa de servicios para separar lógica de negocio.

AuthService

- login
- logout
- registro con dirección

ProductService

- crear producto
- actualizar
- eliminar
- asignar categorías

CategoryService

- CRUD categorías
- soft delete

CartService

- carrito en sesión
- cálculo de total

CheckoutService

- convierte carrito en orden
- valida stock
- usa transacciones
- descuenta stock

OrderService

- crea órdenes
- controla estados válidos

OrderItemService

- guarda productos dentro de la orden

ReviewService

- crear reseñas
- eliminar

WishlistService

- favoritos

FileService

- subida de imágenes
- conversión a WebP

SEGURIDAD

Middleware

AdminMiddleware

Restringe acceso solo a administradores.

Policies

AddressPolicy

- usuario solo ve su dirección
- admin ve todo

OrderPolicy

- usuario solo ve sus pedidos

ReviewPolicy

- usuario o admin puede eliminar

WishlistPolicy

- usuario solo gestiona su wishlist

API REST

El sistema expone una API REST básica para permitir el acceso a datos desde clientes externos.

Las respuestas se devuelven en formato **JSON**, siguiendo buenas prácticas de desarrollo backend.

Rutas API

Las rutas se definen en el archivo:

routes/api.php

Se utilizan controladores específicos dentro del namespace:

App\Http\Controllers\Api

ProductController

Este controlador permite acceder al catálogo de productos mediante la API.

Obtener todos los productos

GET /api/products

Implementación:

```
public function index()
{
    return response()->json(Product::all(), 200);
}
```

- ✓ Retorna todos los productos en formato JSON
- ✓ Código HTTP: **200 OK**

Obtener un producto específico

GET /api/products/{product}

Implementación:

```
public function show(Product $product)
{
    return response()->json($product, 200);
}
```

- ✓ Utiliza **Route Model Binding**
- ✓ Retorna un producto específico

OrderController

Permite obtener los pedidos del usuario autenticado.

Obtener pedidos del usuario

GET /api/orders

Implementación:

```
public function index()
{
    return response()->json(Auth::user()->orders, 200);
}
```

- ✓ Retorna los pedidos del usuario autenticado
- ✓ Usa relación Eloquent (**orders**)

Autenticación

La ruta `/orders` está protegida mediante middleware:

```
Route::middleware('auth.basic')->get('/orders', ...);
```

- ✓ Se utiliza **HTTP Basic Auth**
- ✓ Solo usuarios autenticados pueden acceder

Configuración de la aplicación

En el archivo:

`bootstrap/app.php`

Se configura el sistema de middlewares personalizados.

Alias de middleware

```
$middleware->alias([  
  
    'admin' => AdminMiddleware::class,  
  
]);
```

Permite usar el middleware como:

```
->middleware('admin')
```

Restringe rutas solo a administradores

Seguridad

- Uso de autenticación en rutas sensibles
- Separación entre rutas públicas y protegidas
- Uso de middleware personalizado (`AdminMiddleware`)

Buenas prácticas aplicadas

- ✓ Uso de controladores separados para API
- ✓ Respuestas en formato JSON
- ✓ Uso de códigos HTTP correctos
- ✓ Uso de Route Model Binding
- ✓ Protección de rutas con middleware
- ✓ Separación entre lógica web y API

Vistas (Blade Templates)

El sistema utiliza el motor de plantillas **Blade** de Laravel para la construcción de la interfaz de usuario.

Se implementa una estructura reutilizable basada en **layouts**, permitiendo mantener consistencia visual en toda la aplicación.

Layout principal

Archivo:

resources/views/layouts/layout.blade.php

Este layout define la estructura base de todas las páginas.

Características

- Inclusión de header y footer reutilizables
- Uso de Bootstrap para estilos
- Secciones dinámicas con Blade:
 - `@yield('title')`
 - `@yield('content')`
- Uso de stacks:
 - `@stack('styles')`
 - `@stack('scripts')`

Ejemplo:

```
@include('layouts.header')
```

```
<main>
```

```
    @yield('content')
```

```
</main>
```

```
@include('layouts.footer')
```

Header (Barra de navegación)

Archivo:

resources/views/layouts/header.blade.php

Funcionalidad

El header cambia dinámicamente según el tipo de usuario:

Usuario común

- Acceso a:
 - Inicio
 - Productos
 - Sobre nosotros
 - Envíos
- Íconos:
 - Carrito (con contador)
 - Favoritos (wishlist)
- Botones:
 - Login / Registro

Administrador

- Acceso a:
 - Panel
 - Categorías
 - Productos
 - Pedidos
 - Usuarios
- Indicador visual de rol admin
- Botón de logout

Lógica implementada

Auth::check()

Auth::user()->es_admin

- ✓ Renderizado condicional según autenticación
- ✓ Conteo dinámico de carrito y wishlist

Footer

Archivo:

resources/views/layouts/footer.blade.php

Funcionalidad

- Footer dinámico según la página
- Información de la marca
- Links de navegación
- Año automático:

```
{{ date('Y') }}
```

Home (Landing Page)

Archivo:

resources/views/home_landing/home.blade.php

Secciones

- Hero principal (presentación de marca)
- Categorías de productos
- Información nutricional (elemento distintivo)
- Beneficios del servicio
- CTA (Call To Action)

Características

- ✓ Navegación por categorías
- ✓ Diseño orientado a marketing
- ✓ Uso de rutas dinámicas:

```
route('catalog', ['category' => 'Snacks'])
```

Página "Sobre Nosotros"

Archivo:

resources/views/pages/about.blade.php

Contenido

- Presentación del proyecto
 - Misión
 - Beneficios de productos sin TACC
 - Testimonios de usuarios
- ✓ Enfoque informativo
 - ✓ Refuerza confianza del usuario

Página "Envíos"

Archivo:

resources/views/pages/envios.blade.php

Contenido

- Información de envíos
- Tiempos de entrega
- Costos
- CTA hacia catálogo

- ✓ Mejora la experiencia del usuario
- ✓ Reduce dudas antes de comprar

Buenas prácticas aplicadas

- ✓ Uso de layouts reutilizables
- ✓ Separación de componentes (header/footer)
- ✓ Renderizado condicional según usuario
- ✓ Uso de rutas dinámicas (`route()`)
- ✓ Uso de assets centralizados (`asset()`)
- ✓ Código limpio y organizado

Vistas de Autenticación

El sistema implementa vistas personalizadas de autenticación utilizando Blade, permitiendo a los usuarios registrarse e iniciar sesión.

Estas vistas se encuentran en:

resources/views/auth/

Login

Archivo:

auth/login.blade.php

Descripción

Vista encargada de permitir a los usuarios ingresar al sistema mediante email y contraseña.

Características

- ✓ Formulario con método POST
- ✓ Protección CSRF:

@csrf

Validación de errores:

@error('email')

```
<p class="error-msg">{{ $message }}</p>
```

@enderror

Uso de valores anteriores:

```
value="{{ old('email') }}"
```

Estructura

La vista se divide en dos paneles:

Panel izquierdo (branding)

- * Logo de la marca (Vivra)
- * Mensaje de bienvenida
- * Beneficios de tener cuenta:
 - * Seguimiento de pedidos
 - * Direcciones guardadas
 - * Checkout rápido

Panel derecho (formulario)

- * Campo email
- * Campo contraseña
- * Botón de login
- * Botón cancelar
- * Link a registro

Ruta utilizada

`route('login')`

Registro

Archivo:

`auth/register.blade.php`

Descripción

Permite a nuevos usuarios crear una cuenta en el sistema.

Características

- ✓ Formulario POST
- ✓ Protección CSRF
- ✓ Validaciones por campo
- ✓ Confirmación de contraseña

Campos del formulario

- * Nombre
- * Email
- * Dirección
- * Ciudad
- * Provincia
- * Código postal
- * Contraseña
- * Confirmación de contraseña

Ejemplo de campo

```
<input type="text" name="nombre" value="{{ old('nombre') }}" required>
```

Manejo de errores

```
@error('nombre')
```

```
<p class="error-msg">{{ $message }}</p>
```

```
@enderror
```

Estructura visual

Igual que login:

Panel izquierdo

- * Branding
- * Mensaje motivacional

* Beneficios

Panel derecho

* Formulario completo de registro

Ruta utilizada

`route('register.post')`

Estilos

Ambas vistas utilizan un archivo CSS compartido:

`asset('css/login/login.css')`

✓ Diseño moderno

✓ Layout dividido

✓ Responsive

Buenas prácticas aplicadas

✓ Separación de vistas de autenticación

✓ Reutilización de estilos

✓ Validación de formularios

✓ Uso de Blade directives (@csrf, @error)

✓ Persistencia de datos (old())

✓ Navegación clara entre login y registro

Módulo Store (Tienda)

El sistema cuenta con un módulo de tienda donde los usuarios pueden:

- ✓ Ver productos
- ✓ Filtrar catálogo
- ✓ Agregar al carrito
- ✓ Comprar
- ✓ Guardar favoritos
- ✓ Dejar reseñas

1. Catálogo de productos

Archivo:

store/catalog.blade.php

Funcionalidad

Permite visualizar todos los productos disponibles con:

- * Nombre
- * Precio
- * Imagen
- * Categorías
- * Stock

Filtros

- ✓ Por categoría (dinámico desde DB)

- ✓ Por precio (rangos)

`request('category')`

`request('price')`

Acciones

- ✓ Ver detalle del producto

- ✓ Agregar al carrito

- ✓ Seleccionar cantidad

Paginación

`$products->lastPage()`

- ✓ Navegación entre páginas

- ✓ Mantiene filtros activos

2. Carrito de compras

Archivo:

`store/cart.blade.php`

Funcionalidad

Permite al usuario gestionar productos agregados:

- ✓ Ver productos

- ✓ Cambiar cantidades

✓ Eliminar productos

✓ Ver total

Acciones

* Disminuir cantidad

* Aumentar cantidad

* Eliminar producto

route('cart.increment')

route('cart.decrement')

route('cart.remove')

Resumen de compra

✓ Subtotal por producto

✓ Total general

✓ Cantidad total de productos

Checkout

* Usuario NO logueado → abre login

* Usuario logueado → procesa compra

route('checkout.process')

3. Checkout exitoso

Archivo:

checkout/success.blade.php

Funcionalidad

Se muestra luego de una compra exitosa.

- ✓ Número de pedido
- ✓ Email del usuario
- ✓ Confirmación visual

Acciones

- * Ver detalle del pedido
- * Volver al catálogo

4. Detalle de producto

Archivo:

store/show.blade.php

Funcionalidad

Muestra información completa del producto:

- ✓ Imagen
- ✓ Nombre
- ✓ Descripción
- ✓ Precio

✓ Stock disponible

Acciones

✓ Comprar ahora

✓ Agregar al carrito

✓ Agregar a favoritos

Wishlist

* Invitado → pide login

* Usuario → guarda/quita favorito

route('wishlist.toggle')

Reseñas

✓ Usuarios logueados pueden:

* Calificar (1 a 5 estrellas)

* Escribir comentarios

✓ Se muestran todas las reseñas:

\$product->reviews

El autor puede eliminar su reseña

Productos relacionados

- ✓ Se muestran productos sugeridos

\$relatedProducts

5. Wishlist (Favoritos)

Archivo:

store/wishlist.blade.php

Funcionalidad

Permite guardar productos favoritos.

- ✓ Lista de productos guardados
- ✓ Acceso rápido al producto
- ✓ Eliminación de favoritos

Acciones

route('wishlist.destroy')

Buenas prácticas aplicada

Panel de Administración

Archivo:

dashboard_admin/index.blade.php

Funcionalidad

Es el panel principal para usuarios con rol administrador.

Permite gestionar todo el sistema desde un solo lugar.

Bienvenida personalizada

```
{{ Auth::user()->nombre }}
```

- ✓ Muestra el nombre del admin logueado
- ✓ Refuerza experiencia personalizada

Resumen del sistema

```
$stats['categories']
```

```
$stats['products']
```

```
$stats['users']
```

- ✓ Cantidad de categorías
- ✓ Cantidad de productos
- ✓ Cantidad de usuarios

El admin puede navegar directamente a:

Categorías

```
route('categories.index')
```

```
route('categories.create')
```

- ✓ Ver categorías
- ✓ Crear nuevas

Productos

route('products.index')

route('products.create')

✓ Administrar catálogo

✓ Crear productos

Usuarios

route('usuarios.index')

route('usuarios.create')

✓ Ver usuarios

✓ Crear usuarios

Pedidos

route('admin.orders.index')

✓ Gestionar pedidos

✓ Controlar estados

Módulo de Pedidos (Usuario)

1. Historial de pedidos

Archivo:

orders/index.blade.php

Funcionalidad

Permite al usuario ver todas sus compras.

Información mostrada

✓ ID del pedido

✓ Estado

✓ Fecha

✓ Total

`$order->status->value`

`$order->created_at`

`$order->total`

Estados del pedido

Ejemplo:

* pendiente

* pagado

* enviado

* entregado

* cancelado

Acción

`route('orders.show', $order)`

✓ Ver detalle del pedido

Estado vacío

✓ Mensaje si no hay compras

✓ Botón para ir al catálogo

2. Detalle del pedido

Archivo:

orders/show.blade.php

Funcionalidad

Muestra toda la información de una compra específica.

Productos del pedido

`$order->items`

✓ Nombre del producto

✓ Precio unitario

✓ Cantidad

✓ Subtotal

✓ Manejo de producto eliminado:

`$item->product ? $item->product->name : 'Producto Eliminado'`

Resumen de compra

✓ Estado actual

✓ Total final

Timeline de estados

Estados:

['pendiente', 'pagado', 'enviado', 'entregado']

- ✓ Muestra progreso del pedido
- ✓ Marca estado actual
- ✓ Diferencia entre completado / actual / futuro

Pedido cancelado

- ✓ Manejo especial visual
- ✓ Estado único

Módulo Productos (ADMIN)

create.blade.php

Función: crear productos

- ✓ Formulario con:
 - * nombre
 - * descripción
 - * precio
 - * stock
 - * imagen
 - * categorías (checkbox multiple)

✓ Usa:

`enctype="multipart/form-data"`

permite subir imágenes

✓ Validaciones:

`@error('campo')`

Categorías:

`name="categories[]"`

relación muchos a muchos

`edit.blade.php`

Función: editar producto

✓ Precarga datos:

`old('name', $product->name)`

Imagen actual:

`asset('storage/' . $product->image)`

Categorías seleccionadas:

`$product->categories->pluck('id')`

`index.blade.php`

Función: listar productos

✓ Muestra:

* imagen

* nombre

* descripción

* precio

* stock

✓ Acciones:

* editar

* eliminar

✓ Estado stock:

`$product->stock <= 0`

muestra sin stock

Módulo Perfil (USUARIO)

index.blade.php

Función: panel del usuario

✓ Muestra:

* nombre

* email

✓ Acciones:

- * editar perfil
- * cambiar contraseña
- * ver pedidos
- * ver favoritos

edit.blade.php

Función: editar datos

✓ Datos:

- * nombre
- * email
- * dirección
- * ciudad
- * provincia
- * código postal

password.blade.php

Función: cambiar contraseña

✓ Campos:

- * contraseña actual
- * nueva
- * confirmación

Módulo Usuarios (ADMIN)

create.blade.php

Crear usuarios

✓ Campos:

* nombre

* email

* contraseña

* checkbox admin

name="es_admin"

define rol

edit.blade.php

Editar usuario

✓ Puede:

* cambiar datos

* cambiar rol (admin / cliente)

index.blade.php

Listado de usuarios

✓ Separación:

* Administradores

* Clientes

\$admins

\$clients

✓ Acciones:

* editar

* eliminar

Módulo Categorías (ADMIN)

Carpeta:

resources/views/categories

1. Crear categoría

Archivo: create.blade.php

Qué hace

✓ Permite crear una nueva categoría

route('categories.store')

Guarda en la base de dato

@csrf

Seguridad obligatoria en Laravel

old('name')

Mantiene el valor si hay error

@error('name')

Muestra validaciones

2. Editar categoría

Archivo: edit.blade.php

Qué hace

✓ Permite modificar una categoría existente

Claves

@method('PUT')

Para actualizar (REST)

old('name', \$category->name)

old('name', \$category->name)

Precarga datos

3. Listado de categorías

Archivo: index.blade.php

Qué hace

✓ Muestra todas las categorías

✓ Permite editar / eliminar / restaurar

Soft Delete

`$category->trashed()`

Esto significa que NO se borra definitivamente

Se puede restaurar

Acciones

✓ Editar

✓ Eliminar

✓ Restaurar

`route('categories.restore')`

Módulo Pedidos (ADMIN)

Carpeta:

`resources/views/admin/orders`

1. Listado de pedidos

Archivo: `index.blade.php`

Qué hace

✓ Muestra todos los pedidos

✓ Con información completa:

* cliente

* email

* fecha

* total

* estado

Estado visual

`$order->status->value`

Usa ENUM o constante

Acción

`route('admin.orders.show')`

Ver detalle del pedido

2. Cancelar pedido

Archivo: `cancel.blade.php`

Qué hace

✓ Confirma antes de cancelar

✓ Evita errores del admin

`<input type="hidden" name="status" value="cancelado">`

Cambia estado

3. Detalle del pedido

Archivo: `show.blade.php`

Información del cliente

`$order->usuario`

✓ nombre

✓ email

✓ fecha

✓ total

Dirección

\$order->usuario->addresses->first()

Relación usuario → direcciones

Timeline de estados

\$states = ['pendiente', 'pagado', 'enviado', 'entregado'];

✓ Muestra progreso

✓ Marca estado actual

✓ Permite avanzar estados

Lógica inteligente

✓ Completado

✓ Siguiendo

✓ Futuro

Cancelación

✓ Bloquea cambios si está cancelado

✓ Muestra mensaje especial

Productos del pedido

\$order->items

- ✓ producto
- ✓ precio
- ✓ cantidad
- ✓ subtotal

Estructura de Vistas (Blade Templates)

El sistema utiliza el motor de plantillas Blade de Laravel para estructurar las vistas de forma reutilizable y modular..

Configuración de Rutas

El archivo `web.php` define todas las rutas del sistema web, organizadas según el tipo de usuario y funcionalidad.

Rutas para usuarios no autenticados (Guest)

```
Route::middleware('guest')->group(function () {
```

Función

Permiten el acceso a login y registro solo a usuarios que **no están logueados**.

Rutas

- `GET /login` → muestra el formulario de login
- `POST /login` → procesa el login
- `GET /register` → muestra el formulario de registro
- `POST /register` → procesa el registro

Rutas públicas

Accesibles para cualquier usuario.

Página principal

```
Route::get('/', function () {  
    return view('home_landing.home');  
})->name('home');
```

Páginas informativas

- [/about](#) → información del sitio
- [/envios](#) → información de envíos

Catálogo

```
Route::get('/catalog', [CatalogController::class, 'index'])->name('catalog');
```

Muestra todos los productos disponibles.

Detalle de producto

```
Route::get('/products/{product}', [StoreController::class, 'show'])->name('products.show');
```

Permite ver un producto específico.

Carrito de compras

Rutas

- Ver carrito → [/cart](#)
- Agregar producto → [/cart/add/{product}](#)
- Incrementar cantidad → [/cart/increment/{product}](#)
- Decrementar cantidad → [/cart/decrement/{product}](#)
- Eliminar producto → [/cart/remove/{product}](#)

Función

Permiten gestionar los productos seleccionados antes de la compra.

Rutas protegidas (usuario autenticado)

```
Route::middleware('auth')->group(function () {
```

Checkout

- Procesar compra → [POST /checkout](#)
- Pantalla de éxito → [GET /checkout/success/{order}](#)

Wishlist (favoritos)

- Ver favoritos → `/wishlist`
- Agregar/quitar → `/wishlist/toggle/{product}`
- Eliminar → `/wishlist/{product}`

Reviews (reseñas)

- Crear reseña → `POST /products/{product}/reviews`
- Eliminar reseña → `DELETE /reviews/{review}`

Pedidos del usuario

- Ver pedidos → `/mis-pedidos`
- Ver detalle → `/mis-pedidos/{order}`

Perfil

- Ver perfil → `/profile`
- Editar perfil → `/profile/edit`
- Actualizar datos → `PUT /profile`
- Cambiar contraseña → `/profile/password`

Logout

`Route::post('/logout', [AuthController::class, 'logout'])->middleware('auth');`

Rutas de administración

`Route::middleware(['auth', 'admin'])->prefix('admin')->group(function () {`

Función

Solo accesibles para usuarios con rol **admin**.

Panel de administración

- `/admin` → dashboard principal
-

Gestión de categorías

- Listar → `/admin/categories`
 - Crear → `/admin/categories/create`
 - Guardar → `POST /admin/categories`
 - Editar → `/admin/categories/{id}/edit`
 - Actualizar → `PUT`
 - Eliminar → `DELETE`
 - Restaurar → `PATCH`
-

Gestión de productos

- Listar → `/admin/products`
 - Crear → `/admin/products/create`
 - Guardar → `POST`
 - Editar → `/admin/products/{id}/edit`
 - Actualizar → `PUT`
 - Eliminar → `DELETE`
-

Gestión de usuarios

- Listar → `/admin/usuarios`
 - Crear → `/admin/usuarios/create`
 - Guardar → `POST`
 - Editar → `/admin/usuarios/{id}/edit`
 - Actualizar → `PUT`
 - Eliminar → `DELETE`
-

Gestión de pedidos

- Listar pedidos → `/admin/orders`
- Ver detalle → `/admin/orders/{order}`
- Actualizar estado → `PUT /status`
- Cancelar pedido → `/cancel`

API REST (

`api.php`

)

Define endpoints para acceso externo al sistema.

Productos

```
Route::get('/products', [ProductController::class, 'index']);
```

```
Route::get('/products/{product}', [ProductController::class, 'show']);
```

Función

Permiten obtener:

- Lista de productos
- Detalle de un producto

Pedidos (protegido)

```
Route::middleware('auth.basic')->get('/orders', [OrderController::class, 'index']);
```

Base de Datos – Seeders

Carpeta:

database/seeders

Los **seeders** se utilizan para cargar datos iniciales en la base de datos de forma automática.

Seeder principal:

DatabaseSeeder

Función

Es el punto de entrada que ejecuta todos los seeders del sistema.

Implementación

```
$this->call([  
    UserSeeder::class,  
    CategorySeeder::class,
```

```
ProductSeeder::class,  
CategoryProductSeeder::class,  
AddressSeeder::class,  
]);
```

Orden de ejecución

El orden es importante porque respeta las relaciones:

1. Usuarios
2. Categorías
3. Productos
4. Relación productos-categorías
5. Direcciones

Seeder: Usuarios (

UserSeeder

)

Función

Crea usuarios iniciales del sistema.

Datos creados

- Administrador:
 - Email: admin@gmail.com
 - Contraseña: 1111
 - Rol: admin
- Cliente:
 - Email: cliente@gmail.com
 - Contraseña: 1111
 - Rol: usuario

Características

- Uso de `Hash::make()` para encriptar contraseñas
- Campo `es_admin` para diferenciar roles

Seeder: Categorías (

CategorySeeder

)

Función

Carga las categorías de productos.

Categorías creadas

- Snacks
- Dulces
- Salados
- Harinas
- Congelados

Seeder: Productos (

ProductSeeder

)

Función

Carga el catálogo inicial de productos.

Datos incluidos

Cada producto contiene:

- Nombre
- Descripción
- Precio
- Stock
- Imagen

Características

- Más de 30 productos precargados
- Imágenes asociadas mediante rutas ([storage/products](#))

Seeder: Relación Categoría–Producto (

CategoryProductSeeder

)

Función

Asigna productos a sus respectivas categorías.

Lógica implementada

1. Busca las categorías por nombre
2. Define listas de productos por categoría
3. Relaciona cada producto con su categoría

Tipo de relación

Muchos a muchos (**many-to-many**)

```
$product->categories()->attach($category->id);
```

Seeder: Direcciones (

AddressSeeder

)

Función

Asigna direcciones a usuarios existentes.

Datos creados

- Dirección para admin
- Dirección para cliente

Campos

- usuario_id
- dirección
- ciudad

- provincia
- código postal

Almacenamiento de Archivos

Carpeta:

storage/app/public

Se utiliza para almacenar archivos accesibles públicamente.

Carpeta:

products

Función

Contiene las imágenes de los productos.

Uso en el sistema

Los productos guardan la ruta de la imagen:

'image' => 'products/archivo.webp'

Carpeta:

logos

Función

Contiene imágenes del logo del sistema.

Uso

Se utiliza en:

- Navbar
- Landing page
- Panel de autenticación

Comando para ejecutar seeders

php artisan migrate:fresh --seed

¿Qué hace?

- Borra la base de datos
- Ejecuta migraciones
- Inserta todos los datos iniciales

Estilos del Sistema (CSS)

Carpeta:

`public/css`

Función

Contiene todos los archivos de estilos (CSS) utilizados en la aplicación para definir el diseño visual, la estructura y la experiencia de usuario.

Organización de estilos

Los estilos están organizados en subcarpetas según la funcionalidad del sistema, lo que permite mantener un código ordenado y escalable.

Dashboard de Administración

`dashboard_admin`

Contiene los estilos del panel de administración.

Archivos

- `categories.css` → estilos para gestión de categorías
- `forms.css` → estilos generales de formularios del admin
- `index.css` → estilos del dashboard principal
- `orders.css` → estilos para gestión de pedidos
- `products.css` → estilos para gestión de productos

Landing Page

`home_landing`

- `home.css` → estilos de la página principal (home)

Layouts (estructura general)

layouts

- `header.css` → estilos del encabezado
- `footer.css` → estilos del pie de página

Autenticación

login

- `login.css` → estilos del formulario de inicio de sesión
- `register.css` → estilos del registro de usuarios

Páginas informativas

pages

- `about.css` → estilos de la página “Sobre nosotros”

Tienda (Store)

store

- `cart.css` → estilos del carrito de compras
- `catalog.css` → estilos del catálogo de productos
- `customer-orders.css` → estilos de pedidos del usuario
- `order-shows.css` → estilos del detalle de pedidos
- `show.css` → estilos del detalle de producto
- `wishlists.css` → estilos de la lista de favoritos

Archivos globales

Ubicados directamente en `public/css`

- `base.css` → estilos generales del sistema (reset, tipografía, etc.)
- `checkout.css` → estilos del proceso de compra
- `profile.css` → estilos del perfil de usuario

API y Testing con Postman

Archivo: `Api.postman_collection.json`

Descripción

Se incluye una colección de Postman que permite probar y validar los endpoints de la API desarrollada en el sistema.

Esta colección agrupa distintas requests HTTP que simulan el comportamiento real de un cliente consumiendo la API.

Objetivo

- Verificar el correcto funcionamiento de los endpoints
- Validar respuestas del servidor
- Simular consumo externo de la API
- Facilitar pruebas sin necesidad del frontend

Endpoints incluidos

Obtener pedidos

- **Método:** GET
- **URL:** `/api/orders`
- **Autenticación:** Basic Auth

Descripción

Obtiene el listado de pedidos del sistema en formato JSON.

Ejemplo de respuesta

```
[
  {
    "id": 1,
    "usuario_id": 2,
    "total": "3600.00",
    "status": "cancelado"
  }
]
```

]

Validación

- Se verifica que el status sea **200 OK**

Obtener un producto

- **Método:** GET
- **URL:** `/api/products/{id}`

Descripción

Devuelve la información de un producto específico.

Ejemplo de respuesta

```
{  
  "id": 1,  
  "name": "Fideos sin gluten",  
  "price": "2500.00",  
  "stock": 19  
}
```

Obtener todos los productos

- **Método:** GET
- **URL:** `/api/products`

Descripción

Devuelve el catálogo completo de productos en formato JSON.

Características

- Incluye nombre, precio, stock e imagen
- Se utiliza para consumo externo o apps

Tests automáticos

Dentro de la colección se incluyen tests en JavaScript que validan las respuestas.

Ejemplo:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

Función

- Verificar que la API responde correctamente
- Detectar errores automáticamente
- Asegurar estabilidad del sistema

Autenticación

Para el endpoint de pedidos ([/api/orders](#)) se utiliza:

- **Basic Auth**
- Email y contraseña del usuario

Esto garantiza que solo usuarios autenticados puedan acceder a información sensible.

Uso

1. Importar el archivo [.json](#) en Postman
2. Ejecutar las requests
3. Verificar respuestas y tests

La colección de Postman permite testear la API de forma rápida y organizada, asegurando que los endpoints funcionen correctamente y cumplan con los requisitos del sistema.

